

Ejercicio Práctico: Configuración de Ingress en Kubernetes

1. Título y Objetivo

Título: Enrutamiento de tráfico HTTP con Kubernetes Ingress y NGINX Ingress Controller

Objetivo: Configurar un recurso Ingress que enrute solicitudes HTTP entrantes hacia dos servicios backend distintos (`community-api-svc` y `enterprise-api-svc`) en función del path de la URL, usando el controlador NGINX ya desplegado en el clúster.

Habilidad que se evalúa: - Definir y aplicar manifiestos de tipo Ingress en Kubernetes - Comprender el enrutamiento basado en paths con `pathType: Prefix` - Usar anotaciones de NGINX para reescritura de rutas (`rewrite-target`) - Verificar el correcto funcionamiento del Ingress con `kubectl` y `curl`

2. Conceptos Clave

Ingress

Un Ingress es un recurso de Kubernetes que gestiona el acceso HTTP/HTTPS externo hacia los servicios dentro del clúster. Actúa como una capa de enrutamiento L7 (capa de aplicación), permitiendo definir reglas basadas en host, path o ambas.

Sin un Ingress, cada servicio que necesita exposición externa requeriría su propio LoadBalancer o NodePort, lo que es costoso e ineficiente. Con Ingress, un único punto de entrada puede distribuir el tráfico a múltiples servicios.

Cliente HTTP

|
v

```
[ Ingress ] --> /community/ --> community-api-svc:80  
           --> /enterprise/ --> enterprise-api-svc:80
```

IngressController (NGINX)

El recurso Ingress por sí solo no hace nada. Requiere un **Ingress Controller** que lo interprete y configure el proxy real. En este ejercicio se usa el **NGINX Ingress Controller**, uno de los más extendidos en la comunidad.

Componente	Rol
Ingress (recurso)	Declara las reglas de enrutamiento
Ingress Controller	Implementa esas reglas (configura NGINX internamente)
NGINX	

Componente	Rol
	Proxy que recibe el tráfico y lo reenvía al servicio correspondiente

pathType: Prefix vs Exact

El campo pathType define cómo se interpreta el path especificado en la regla.

Valor	Comportamiento
Prefix	Hace match con el path y cualquier subpath. /community/ también coincide con /community/ping, /community/users/1, etc.
Exact	Solo hace match con el path exacto. /community/ no coincide con /community/ping.

En este ejercicio se usa Prefix porque los servicios exponen múltiples endpoints bajo el path base.

Annotations

Las anotaciones (annotations) son metadatos clave-valor que proporcionan instrucciones adicionales al Ingress Controller. No son interpretadas por Kubernetes directamente, sino por el controlador específico (en este caso, NGINX).

rewrite-target

La anotación `nginx.ingress.kubernetes.io/rewrite-target: /` le indica al Ingress Controller que, antes de reenviar la solicitud al servicio backend, reescriba el path eliminando el prefijo.

Sin rewrite-target:

Solicitud entrante: `GET /community/ping`
 Path enviado al pod: `GET /community/ping` ← el servicio no tiene esta ruta

Con rewrite-target: /

Solicitud entrante: `GET /community/ping`
 Path enviado al pod: `GET /ping` ← el servicio sí reconoce esta ruta

Esto es necesario cuando los servicios backend no conocen el prefijo de path del Ingress.

3. Pre-requisitos

Antes de comenzar este ejercicio, lo siguiente debe estar disponible en el clúster:

Servicios backend existentes

Los dos servicios que actúan como backend ya deben existir. Verificar con:

```
kubectl get svc community-api-svc enterprise-api-svc
```

Salida esperada:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
PORT(S) AGE			
community-api-svc	ClusterIP	10.96.x.x	<none> 80/
TCP Xm			
enterprise-api-svc	ClusterIP	10.96.x.x	<none> 80/
TCP Xm			

NGINX Ingress Controller desplegado

El Ingress Controller debe estar corriendo en el clúster. Verificar con:

```
kubectl get pods -n ingress-nginx
```

Salida esperada (al menos un pod en estado Running):

NAME	READY	STATUS
RESTARTS AGE		
ingress-nginx-controller-xxxxxxxxxx-xxxxx	1/1	Running
0 Xm		

DNS configurado

El dominio `api.example.local` debe resolver a la IP del Ingress Controller (o del nodo donde corre). Esto puede verificarse con:

```
curl -v api.example.local
# o en entornos locales, verificar /etc/hosts
cat /etc/hosts | grep api.example.local
```

kubectl configurado

El CLI `kubectl` debe estar configurado con los permisos necesarios para crear recursos de tipo Ingress en el namespace activo:

```
kubectl auth can-i create ingress
# expected: yes
```

4. Pasos para Reproducir

Paso 1: Verificar que los servicios backend existen

```
kubectl get svc community-api-svc enterprise-api-svc
```

Confirmar que ambos servicios existen y exponen el puerto 80.

Paso 2: Verificar que el Ingress Controller está activo

```
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
```

Anotar la IP o hostname del servicio ingress-nginx-controller para usarla en las pruebas.

Paso 3: Crear el archivo YAML del Ingress

Crear el archivo `ing.yaml` con el contenido del manifiesto (ver sección 5). Se puede hacer con cualquier editor:

```
vim ing.yaml
# o
nano ing.yaml
```

Paso 4: Aplicar el manifiesto

```
kubectl apply -f ing.yaml
```

Salida esperada:

```
ingress.networking.k8s.io/api-ingress created
```

Paso 5: Verificar que el Ingress fue creado correctamente

```
kubectl get ingress api-ingress
```

Salida esperada:

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
api-ingress	nginx	api.example.local	<cluster-ip>	80	Xs

Paso 6: Inspeccionar el detalle del Ingress

```
kubectl describe ingress api-ingress
```

Revisar que las reglas, paths y backends están configurados como se espera.

Paso 7: Probar los endpoints con curl

```
curl -s api.example.local/community/ping
curl -s api.example.local/enterprise/ping
```

Ver sección 6 para los resultados esperados.

5. Manifiesto YAML Completo

```
# Versión de la API de Kubernetes para recursos Ingress (estable desde 1.19)
apiVersion: networking.k8s.io/v1

# Tipo de recurso
kind: Ingress

metadata:
  # Nombre único del Ingress en el namespace
  name: api-ingress
```

```

annotations:
  # Indica al NGINX Ingress Controller que reescriba el path de la
  # solicitud
  # antes de reenviarla al servicio backend. El valor "/" significa
  # que el
  # prefijo del path (/community/ o /enterprise/) se elimina, y el
  # servicio
  # recibe solo la parte restante (ej: /ping en lugar de /community/
  # ping).
  nginx.ingress.kubernetes.io/rewrite-target: /

spec:
  rules:
    # Regla para el host api.example.local
    # Solo las solicitudes con este Host header serán procesadas por
    # este Ingress
    - host: api.example.local
      http:
        paths:

          # Primera regla: rutas que comienzan con /community/
          - path: /community/
            # Prefix: el path y cualquier subpath harán match
            # /community/, /community/ping, /community/users/1, etc.
            pathType: Prefix
            backend:
              service:
                # Nombre del servicio Kubernetes al que se reenvía el
                # tráfico
                name: community-api-svc
                port:
                  # Puerto del servicio (no del pod/contenedor, sino del
                  # Service)
                  number: 80

            # Segunda regla: rutas que comienzan con /enterprise/
            - path: /enterprise/
              pathType: Prefix
              backend:
                service:
                  name: enterprise-api-svc
                  port:
                    number: 80

```

6. Comandos de Verificación

Verificar estado del Ingress

```

# Listar el Ingress y ver su ADDRESS (IP asignada)
kubectl get ingress api-ingress

# Ver detalles completos: reglas, backends, anotaciones y eventos
kubectl describe ingress api-ingress

```

```
# Ver el YAML tal como quedó almacenado en el clúster
kubectl get ingress api-ingress -o yaml
```

Probar los endpoints con curl

```
# Endpoint del servicio community
curl -s api.example.local/community/ping
```

Resultado esperado:

```
{"type":"community","response":"OK"}
```

```
# Endpoint del servicio enterprise
curl -s api.example.local/enterprise/ping
```

Resultado esperado:

```
{"type":"enterprise","response":"OK"}
```

Verificar con cabeceras HTTP completas (modo verbose)

```
# Ver la solicitud y respuesta HTTP completa para community
curl -v api.example.local/community/ping

# Ver la solicitud y respuesta HTTP completa para enterprise
curl -v api.example.local/enterprise/ping
```

Verificar logs del Ingress Controller (diagnóstico)

```
# Ver los logs en tiempo real del NGINX Ingress Controller
kubectl logs -n ingress-nginx \
$(kubectl get pods -n ingress-nginx -o name | grep controller) \
--tail=50 -f
```

Tabla resumen de comandos de verificación

Comando	Propósito
kubectl get ingress api-ingress	Confirmar que el Ingress existe y tiene una IP asignada
kubectl describe ingress api-ingress	Ver reglas, backends y posibles errores
curl -s api.example.local/community/ping	Probar enrutamiento al servicio community
curl -s api.example.local/enterprise/ping	Probar enrutamiento al servicio enterprise
kubectl logs -n ingress-nginx ...	Diagnosticar errores en el controlador NGINX

7. Errores Comunes

Error 1: pathType en mayúsculas incorrectas o inválido

Problema: Escribir pathtype, prefix o PREFIX en lugar de Prefix.

```
# INCORRECTO
pathType: prefix

# CORRECTO
pathType: Prefix
```

Los valores válidos son sensibles a mayúsculas: Prefix, Exact, ImplementationSpecific.

Error 2: Falta la anotación rewrite-target

Problema: Sin la anotación, el path completo (incluyendo /community/) se reenvía al servicio backend, que probablemente no tiene esa ruta registrada.

Síntoma: El curl devuelve 404 Not Found del servicio, no del Ingress.

```
# INCORRECTO (falta la anotación)
metadata:
  name: api-ingress

# CORRECTO
metadata:
  name: api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
```

Error 3: Nombre incorrecto del servicio backend

Problema: Un typo en el nombre del servicio en backend.service.name.

Síntoma: El Ingress se crea sin error (Kubernetes no valida que el servicio exista en el momento de la creación), pero las solicitudes devuelven 503 Service Unavailable.

```
# INCORRECTO
name: community-api-svc # falta una 'm'

# CORRECTO
name: community-api-svc
```

Verificar el nombre exacto del servicio antes de escribir el manifiesto:

```
kubectl get svc
```

Error 4: Puerto del servicio incorrecto

Problema: Especificar un puerto que el servicio no expone.

Síntoma: 503 Service Unavailable o el Ingress queda sin endpoints.

```
# INCORRECTO (si el servicio expone el puerto 80, no el 8080)
port:
  number: 8080

# CORRECTO
port:
  number: 80
```

Verificar el puerto del servicio con:

```
kubectl get svc community-api-svc -o jsonpath='{.spec.ports[*].port}'
```

Error 5: apiVersion desactualizada

Problema: Usar `extensions/v1beta1` o `networking.k8s.io/v1beta1`, que fueron deprecadas y eliminadas en Kubernetes 1.22+.

```
# INCORRECTO (obsoleto en Kubernetes >= 1.22)
apiVersion: networking.k8s.io/v1beta1

# CORRECTO
apiVersion: networking.k8s.io/v1
```

Error 6: El host no resuelve al Ingress Controller

Problema: El dominio `api.example.local` no está configurado en DNS o en `/etc/hosts`.

Síntoma: `curl: (6) Could not resolve host: api.example.local`

Solución temporal para pruebas locales:

```
# Obtener la IP del Ingress Controller
kubectl get svc -n ingress-nginx ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'

# Agregar al /etc/hosts (reemplazar <IP> con la IP obtenida)
echo "<IP> api.example.local" | sudo tee -a /etc/hosts
```

8. Tip: Aplicar desde GitHub (técnica usada en este lab)

En lugar de crear el archivo localmente, es posible aplicar un manifiesto directamente desde una URL raw de GitHub. Esta técnica es útil para compartir configuraciones reproducibles o en entornos donde no se quiere copiar archivos manualmente.

Cómo funciona

```
kubectl apply -f https://raw.githubusercontent.com/<usuario>/<repo>/<rama>/<archivo>.yaml
```

kubectl hace un GET HTTP a la URL, descarga el contenido YAML y lo aplica directamente al clúster, exactamente igual que si fuera un archivo local.

Comando usado en este ejercicio

```
kubectl apply -f https://raw.githubusercontent.com/rubenpinzon-sketch/kube-labs/main/ing.yaml
```

Obtener la URL raw desde GitHub

1. Navegar al archivo en GitHub: `github.com/<usuario>/<repo>/blob/<rama>/<archivo>.yaml`
2. Hacer clic en el botón **Raw**
3. Copiar la URL del navegador (comienza con `raw.githubusercontent.com`)

Consideraciones de seguridad

Advertencia: Aplicar manifiestos desde URLs externas sin revisarlos previamente es un riesgo de seguridad. En entornos de producción, siempre revisar el contenido del YAML antes de aplicarlo, o usar repositorios privados con acceso controlado.

Para revisar el contenido antes de aplicar:

```
# Ver el contenido sin aplicar
curl -s https://raw.githubusercontent.com/rubenpinzon-sketch/kube-labs/main/ing.yaml

# Aplicar solo si el contenido es el esperado
kubectl apply -f https://raw.githubusercontent.com/rubenpinzon-sketch/kube-labs/main/ing.yaml
```

Otras variantes útiles

```
# Aplicar todos los archivos YAML de un directorio en GitHub (no soportado directamente)
# En su lugar, usar kustomize o listar los archivos manualmente

# Modo dry-run para ver qué se crearía sin aplicarlo
kubectl apply -f <url> --dry-run=client

# Ver el diff entre lo que existe y lo que se aplicaría
kubectl diff -f <url>
```

9. Referencias

Recurso	URL
Documentación oficial de Ingress	https://kubernetes.io/docs/concepts/services-networking/ingress/
Ingress Controllers disponibles	https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/

Recurso	URL
NGINX Ingress Controller (docs oficiales)	https://kubernetes.github.io/ingress-nginx/
Anotaciones del NGINX Ingress Controller	https://kubernetes.github.io/ingress-nginx/user-guide/nginx-configuration/annotations/
pathType explicado	https://kubernetes.io/docs/concepts/services-networking/ingress/#path-types
API reference: Ingress v1	https://kubernetes.io/docs/reference/kubernetes-api/service-resources/ingress-v1/
Guía de migración de v1beta1 a v1	https://kubernetes.io/docs/reference/using-api/deprecation-guide/#ingress-v122